

PROJECT: Serverless Web Application with AWS Lambda and API Gateway

Services: AWS Lambda, Amazon API Gateway , Amazon DynamoDB , AWS IAM (Identity and Access Management), Amazon CloudWatch

Project Goal: The goal of this project is to create a simple system to store and manage student details using AWS cloud services. It uses serverless technology, so no servers are needed, making the system easy, fast, and cost-effective.

Skills: API integration, serverless architecture, authentication.

Definition: This project is a serverless student management system built using AWS cloud services. It allows users to add and view student details through an API without using or managing servers

Scope of the Project: The scope of this project is to design and implement a **serverless Student Management API** that allows storing and retrieving student details securely. It covers API creation, backend processing, and database integration using AWS services. The project is useful for learning real-world cloud application development and can be extended for use in educational institutions to manage student records efficiently.

Methodologies:

The same student management system can be implemented using the following methods:

- **On-Premise Method**
The application and database are hosted on local servers. This requires high setup cost, regular maintenance, and manual scaling.
- **EC2-Based Method**
The backend runs on Amazon EC2 virtual machines. Although flexible, it requires server management, security updates, and scaling configuration.
- **Container-Based Method (Docker / ECS)**
The application runs inside containers. This method is powerful but complex for small projects and requires extra configuration.

Why Our Method Is Better Than Other Methods

The serverless method is more important because:

- No server setup or maintenance is required
- Automatically scales with user requests
- Lower cost (pay only for usage)
- Faster development and deployment
- High availability managed by AWS

Because of these advantages, serverless architecture is the best and most efficient method for this project compared to traditional approaches.

Services Used in this Project:**1.AWS Lambda****Definition:**

AWS Lambda is a serverless computing service that runs code without managing servers.

Purpose:

It is used to execute backend logic when an API request is received.

Role in This Project:

Lambda processes the GET and POST requests, generates student IDs, and communicates with DynamoDB to store and retrieve student data.

2. Amazon API Gateway**Definition:**

Amazon API Gateway is a service used to create, manage, and expose REST APIs.

Purpose:

It acts as an interface between the user and the backend services.

Role in This Project:

API Gateway receives requests from the client and forwards them to AWS Lambda. It also handles API methods, CORS, and deployment.

3. Amazon DynamoDB**Definition:**

Amazon DynamoDB is a fully managed NoSQL database service.

Purpose:

It stores application data with high speed and scalability.

Role in This Project:

DynamoDB stores student records such as student ID, name, email, and course.

- **AWS IAM (Identity and Access Management)**

Definition:

AWS IAM is a service used to manage permissions and access to AWS resources.

Purpose:

It controls which services can access other services.

Role in This Project:

IAM provides permission for Lambda to access DynamoDB and write logs to CloudWatch.

- **Amazon CloudWatch**

Definition:

Amazon CloudWatch is a monitoring and logging service for AWS resources.

Purpose:

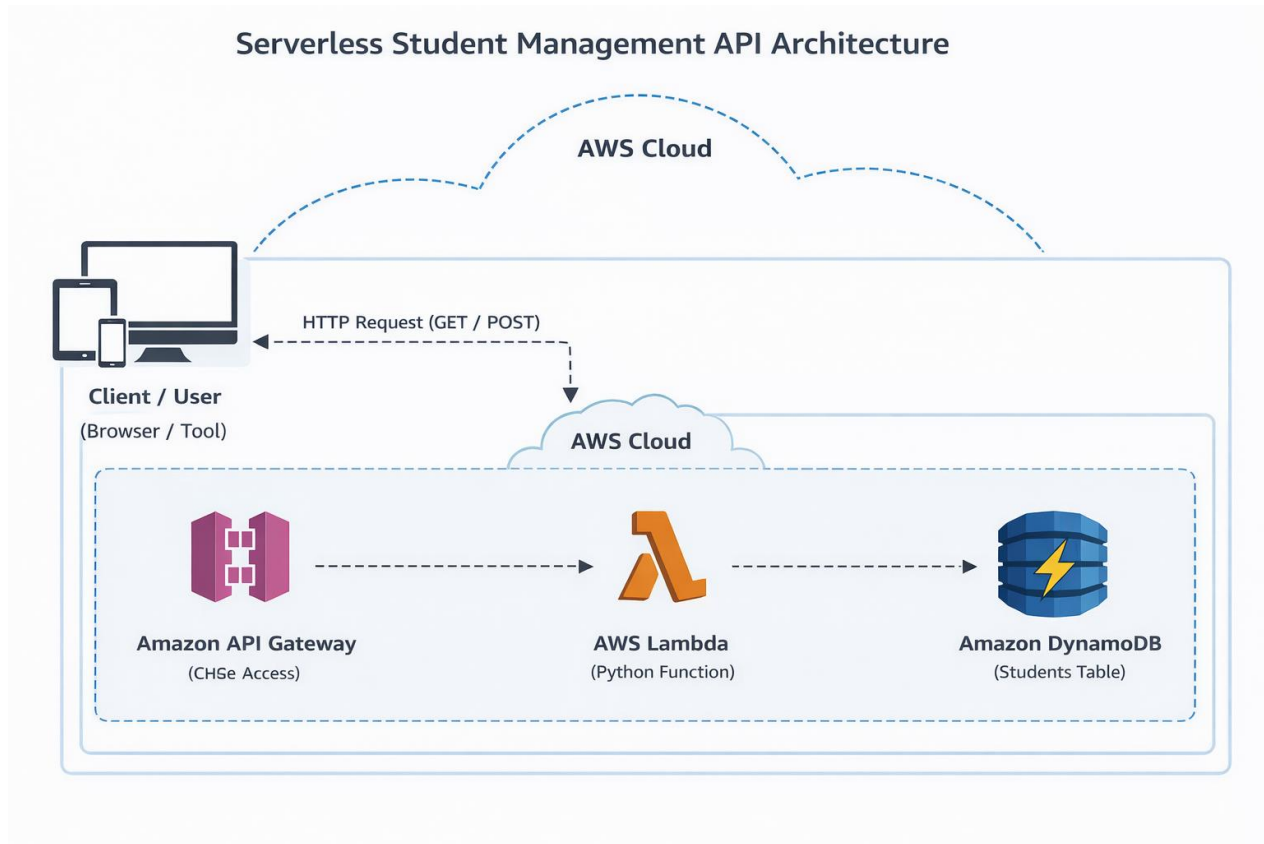
It helps in tracking application performance and errors.

Role in This Project:

CloudWatch stores Lambda execution logs and helps in debugging errors during API execution.

Issues Faced, Causes, and Solutions

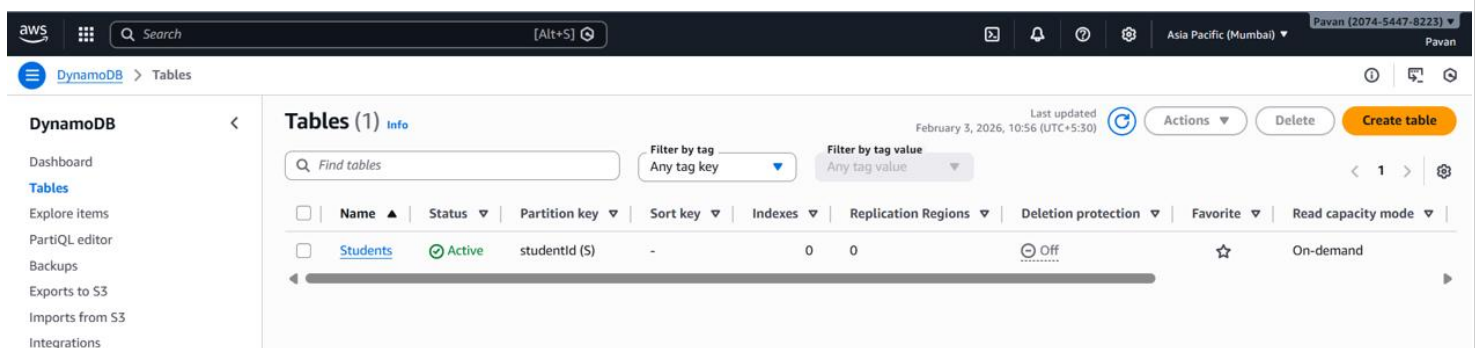
Architecture Diagram



Implementation and Execution of a Serverless Web Application with AWS Lambda and API Gateway.

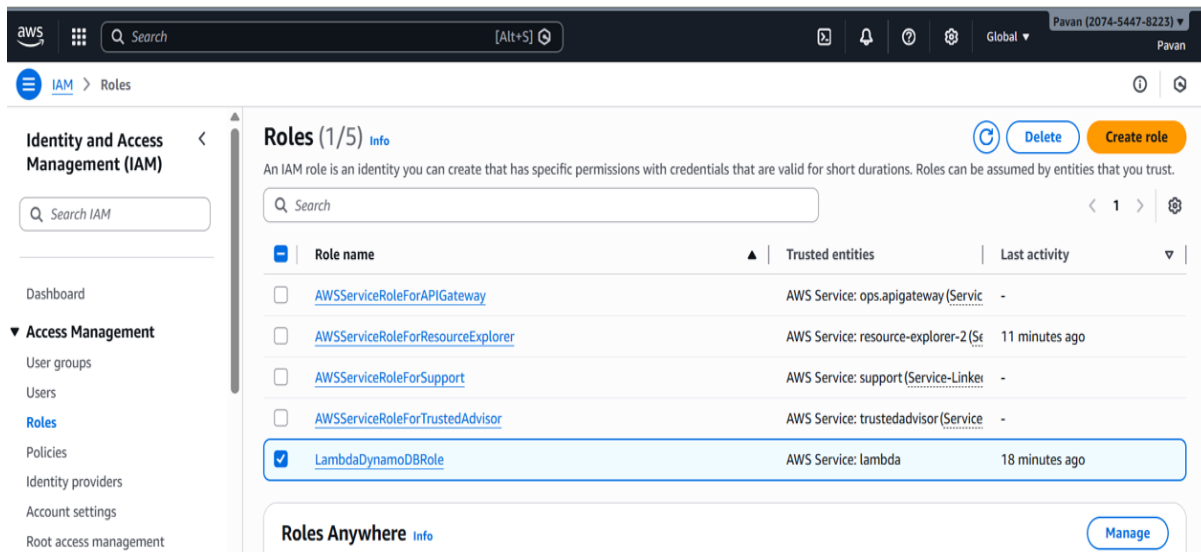
Phase 1. CREATE TABLE

- Open **AWS Console** → **DynamoDB** → **Tables** → **Create Table**
 - **Table name:** Students
 - **Partition key:** studentId
 - **Type:** String
- Click **Create table**
- Wait until **Status = Active**

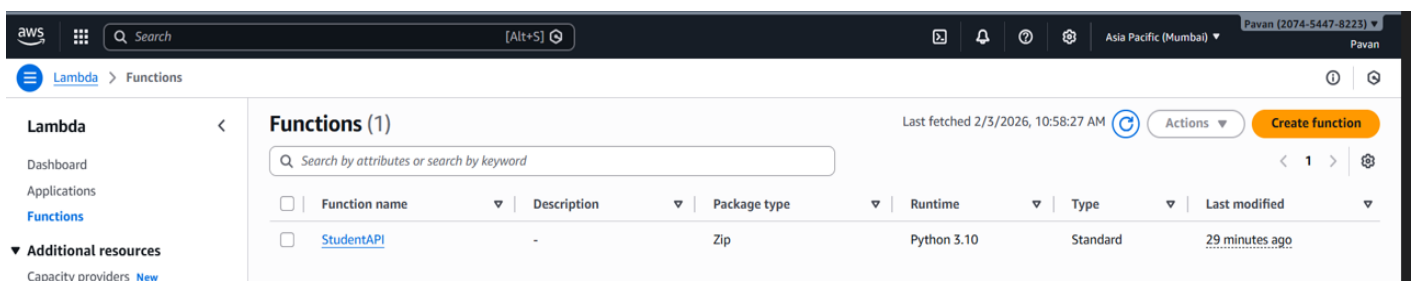


Phase 2: CREATE IAM ROLE FOR LAMBDA

- Go to **IAM** → **Roles** → **Create role**
 - Select **AWS service**
 - Choose **Lambda**
 - Click **Next**
- Attach policies:
 - ☒ **AWSLambdaBasicExecutionRole**
 - ☒ **AmazonDynamoDBFullAccess**
- Role name:
 - LambdaDynamoDBRole**
 - Click **Create role**

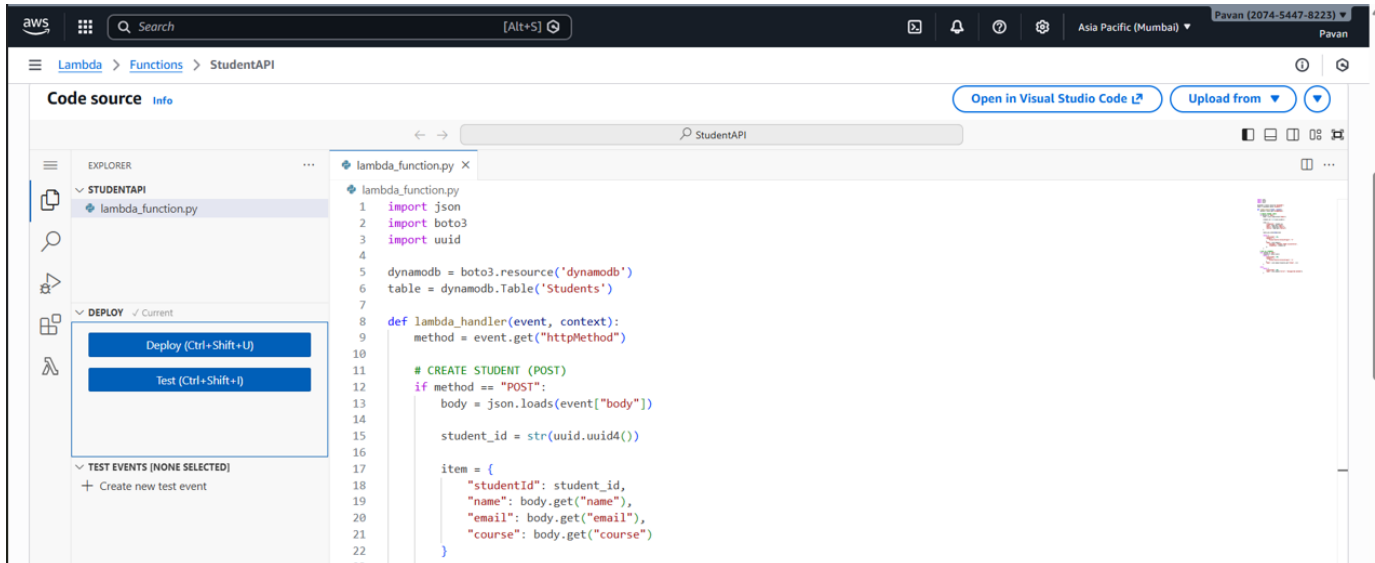
**Phase 3: CREATE LAMBDA**

- Go to **AWS Lambda** → **Create function**
 - Function name:** StudentAPI
 - Runtime:** Python 3.10
 - Execution role:** Use existing role
 - Role:** LambdaDynamoDBRole
- Click **Create function**



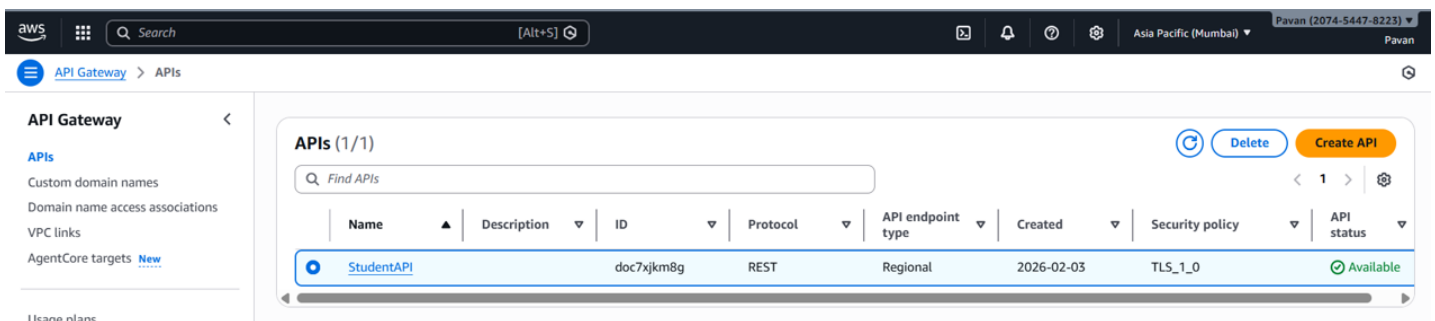
Phase 4 : UPDATE LAMBDA CODE

- Click on Lambda Function
- Replace Lambda code with Python Code
- Click Save
- Click Deploy



Phase 5 : CREATE API GATEWAY:

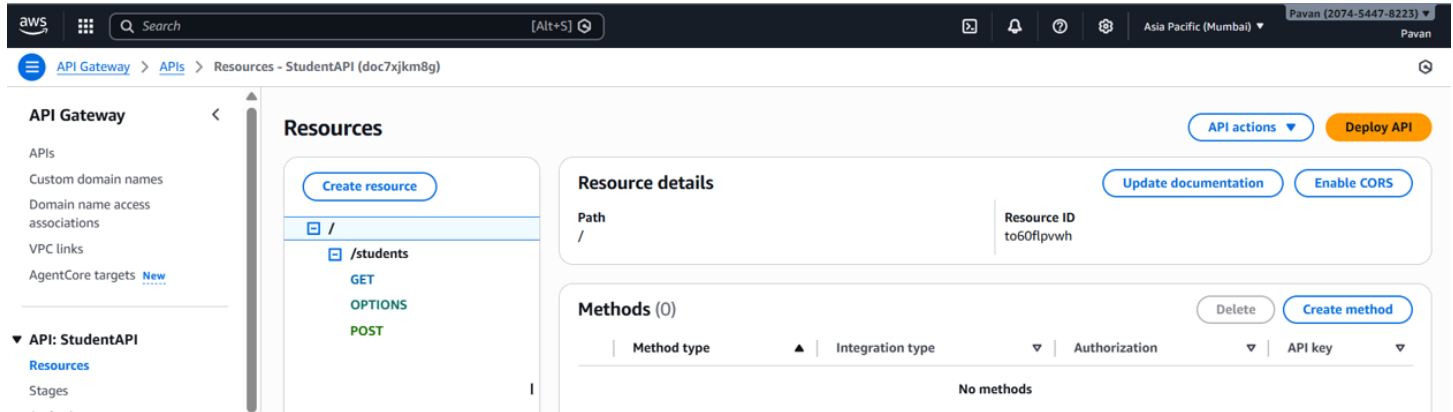
- Go to **API Gateway** → **Create API**
- Select **REST API**
 - Choose **Build**
 - API name:
 - StudentAPI
- Endpoint type: **Regional**
- Click **Create API**



Phase 6: CREATE RESOURCE & METHOD

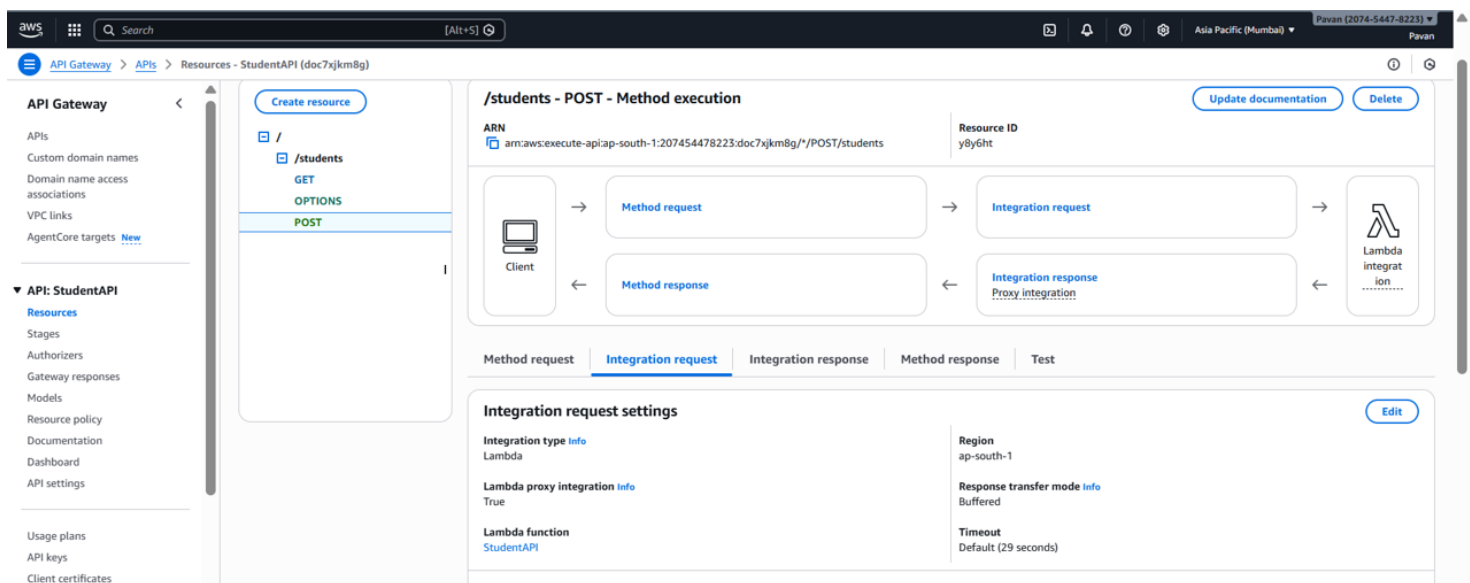
Create Resource

- Select API → **Resources**
 - Click **Create resource**
 - Resource name: students
 - Resource path: /students
- Click **Create resource**



Create POST Method

- Select /students
- Click **Create method** → **POST**
 - Integration type: **Lambda function**
 - Enable **Lambda proxy integration**
 - Select Lambda: StudentAPI
- Click **Create**



Create POST Method

- Select /students
- Click **Create method** → **GET**
 - Integration type: **Lambda function**
 - Enable **Lambda proxy integration**
 - Select Lambda: StudentAPI
- Click **Create**

The screenshot shows the AWS API Gateway console. On the left, the navigation pane is open, showing the hierarchy: API Gateway > APIs > Resources - StudentAPI (doc7xjkm8g). The main panel displays the configuration for the /students GET method. The 'Integration request settings' section is visible, showing the following configuration:

- Integration type:** Lambda
- Lambda proxy integration:** True
- Lambda function:** StudentAPI
- Region:** ap-south-1
- Response transfer mode:** Buffered
- Timeout:** Default (29 seconds)

The 'Method request' tab is selected, showing a diagram of the request flow: Client → Method request → Integration request → Integration response (Proxy integration) → Method response.

STEP 7: ENABLE CORS

- Select /students
- Click **Enable CORS**
 - Check:
 - GET
 - POST
- Click **Save**

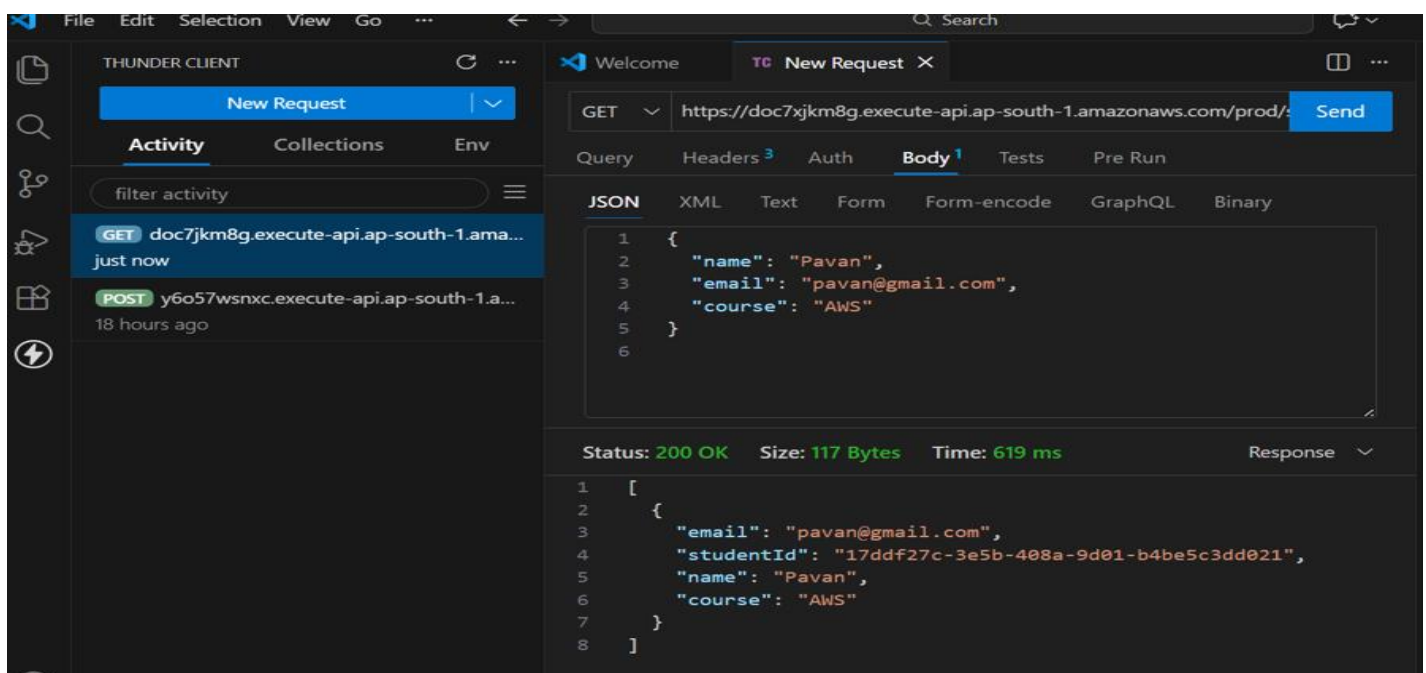
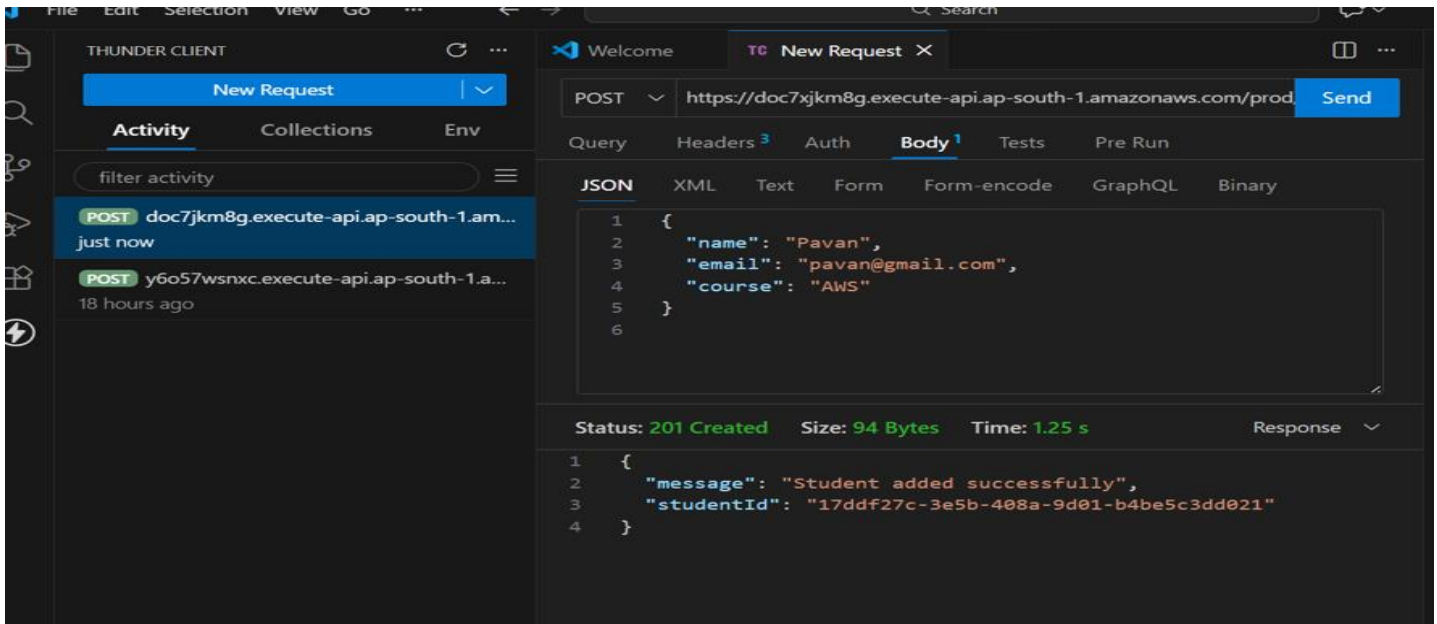
The screenshot shows the 'Enable CORS' configuration page in the AWS API Gateway console. The breadcrumb trail is: API Gateway > APIs > Resources - StudentAPI (doc7xjkm8g) > Enable CORS. The 'CORS settings' section is expanded, showing the following configuration:

- Gateway responses:**
 - ☐ Default 4XX
 - ☐ Default 5XX
- Access-Control-Allow-Methods:**
 - ☒ GET
 - ☒ OPTIONS
 - ☒ POST
- Access-Control-Allow-Headers:**
 - Content-Type,X-Amz-Date,Authorization,X-Api-Key,X-Amz-Security-Token

Phase 8: TEST USING THUNDER CLIENT / POSTMAN

- **Thunder client Browser** → Create New Request
- **Request Details**
- **Method : POST**
 - **URL** : <https://<api-id>.execute-api.ap-south-1.amazonaws.com/prod/students>
 - **Headers** : Content-Type: application/json
 - **Body (JSON)**

```
{
  "name": "Pavan",
  "email": "pavan@gmail.com",
  "course": "AWS"
}
```
- **Method : GET**
 - **URL** : <https://<api-id>.execute-api.ap-south-1.amazonaws.com/prod/students>
 - **Headers** : Content-Type: application/json
 - **Body (JSON)** : (same as POST Body)



Phase 9: VERIFY IN DYNAMODB

- DynamoDB → Tables → Student
- Click **Explore item**
- You will see:

The screenshot shows the AWS DynamoDB console interface. The left sidebar contains navigation options: Dashboard, Tables, Explore items (selected), PartiQL editor, Backups, Exports to S3, Imports from S3, Integrations, Reserved capacity, Settings, DAX, Clusters, Subnet groups, Parameter groups, and Events. The main content area is titled 'Explore items' and shows the 'Students' table. A 'Filter by tag value' section is present, along with a 'Find tables' search bar. The 'Scan' button is selected, and the 'Table - Students' is chosen. The 'Select attribute projection' is set to 'All attributes'. A green status bar indicates 'Completed - Items returned: 1 - Items scanned: 1 - Efficiency: 100% - RCUs consumed: 2'. Below this, a table shows the returned item:

studentId (String)	course	email	name
17ddf27c-3e5b-408a-...	AWS	pavan@gm...	Pavan

The screenshot shows the 'Edit item' view in the AWS DynamoDB console. The title is 'Edit item' and it includes a description: 'You can add, remove, or edit the attributes of an item. You can nest attributes inside other attributes up to 32 levels deep. Learn more'. The 'Form' view is selected. The 'Attributes' section shows a table with columns: Attribute name, Value, Type, and Remove. The attributes are:

Attribute name	Value	Type	Remove
studentId - Partition key	17ddf27c-3e5b-408a-9d01-b4be5c3dd021	String	
course	AWS	String	Remove
email	pavan@gmail.com	String	Remove
name	Pavan	String	Remove

At the bottom, there are buttons for 'Cancel', 'Save', and 'Save and close'.

TroubleShooting

1. 500 Internal Server Error in API Gateway

- **Error Faced:**
API returned 500 Internal Server Error while sending POST request.
- **Cause:**
 - Lambda function had an error while executing
 - Incorrect DynamoDB table name or missing permissions
- **Solution:**
 - Checked **CloudWatch Logs** to find the exact error
 - Corrected the DynamoDB table name in the Lambda code
 - Added proper IAM permissions (dynamodb:PutItem, dynamodb:Scan)

2. ResourceNotFoundException (DynamoDB PutItem)

- **Error Faced :**Requested resource not found while inserting data.
- **Cause:** DynamoDB table name in Lambda code did not exactly match the actual table name
- **Solution:**
 - Verified table name in DynamoDB console
 - Updated the correct table name in Lambda code

3. Changes Not Reflected After Update

- **Error Faced:** API behavior did not change after modifying code or settings.
- **Cause :** API Gateway was not redeployed after changes
- **Solution:**
 - Redeployed API to prod stage
 - Tested again using updated endpoint

Conclusion

This project successfully demonstrates the implementation of a **Serverless Student Management API** using AWS services. By integrating **API Gateway, AWS Lambda, and DynamoDB**, the system allows student data to be stored and retrieved efficiently without managing any servers.

The project helped in understanding **real-time serverless architecture**, API creation, cloud-based databases, and secure access using IAM roles. It also provided hands-on experience in **deploying, testing, and troubleshooting cloud applications**.

Overall, this project shows how modern applications can be built in a **scalable, cost-effective, and efficient** way using AWS, making it suitable for real-world use in educational or organizational systems.

